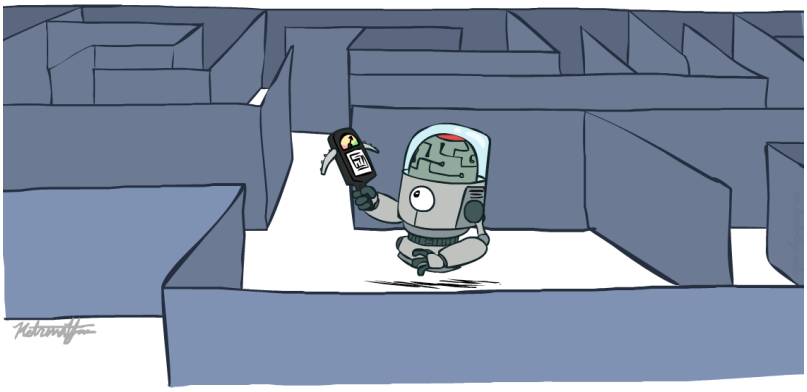
Inteligência Artificial

Busca com Informação



Prof. Dr. Wellington Candeia

[Slides com base em Dan Klein and Pieter Abbeel.]

Agenda

- Busca da Melhor Escolha
- Função Heurística
- Busca Gulosa
- Busca A*
- Heurísticas Admissíveis
- Busca Local e Otimização
- Algoritmo Subida da Encosta (*Hill Climbing*)

Busca de Melhor Escolha (*Best-First*)

- Abordagem Geral de Busca Informada:
  - "Informada" = Baseada em conhecimento de domínio.
  - Busca *Best-First*: nodo é selecionado para expansão baseado em uma função de avaliação f(n)
- Ideia: Estimar a distância até o objetivo.
- Escolhe o nó que aparenta ser o melhor de acordo com f(n).
- Implementação:
  - Fila de prioridades, na qual a chave é f(n).
  - Realizações: Busca Gulosa, Busca A*

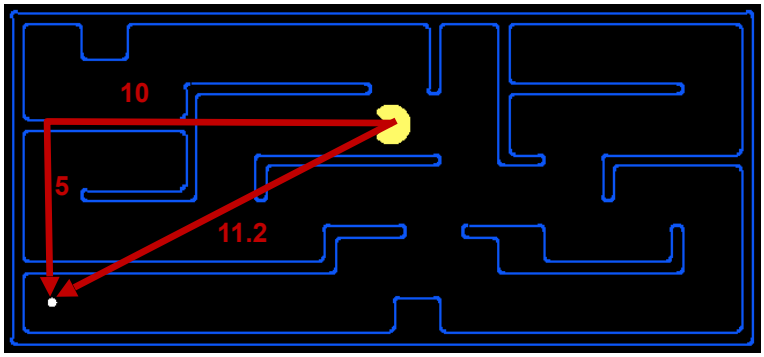
Função Heurística

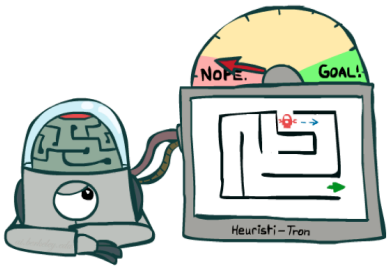
- "Uma regra geral, simplificação, ou estimativa aproximada que reduz ou limita a busca de soluções em domínios que são difíceis e pouco compreendidos".

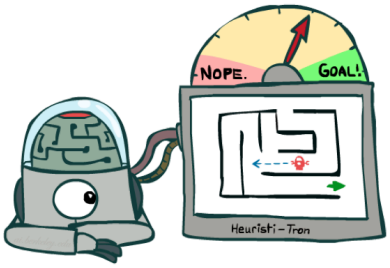
- h(n) = custo estimado do caminho mais curto de n até o nó objetivo.
- Condição Básica: Se n é o objetivo então h(n) = 0.

Busca Heurística

- Uma heurística é:
  - Uma função que *estima* o quanto um estado está perto de um objetivo
  - Projetada para uma busca em particular
  - Exemplos: distância de Manhattan, distância Euclideana







Exemplo: Mapa da Romênia

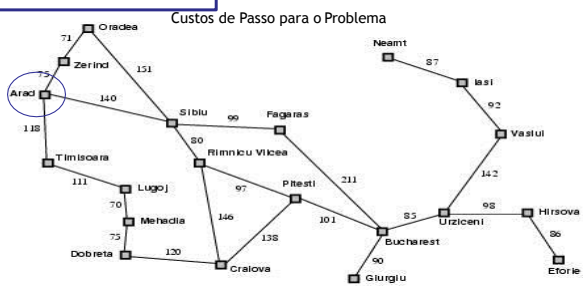
Distâncias em linha reta até Bucareste

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374
Hirsova	151	Urziceni	80

h(x)

- h_{DLR} = heurística da distância em linha reta
- h_{DLR} Não pode ser calculada da descrição do problema em si.
- Neste exemplo: f(n) = h(n)
- Expande o nó "mais perto" do objetivo.
- = Busca Gulosa de Melhor Escolha.

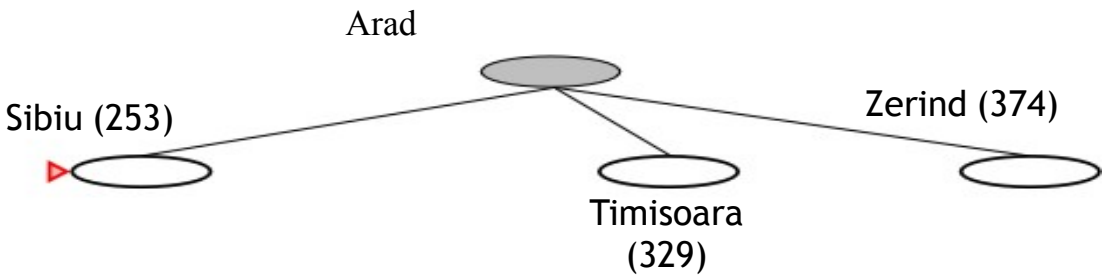
Custos de Passo para o Problema



Busca Gulosa

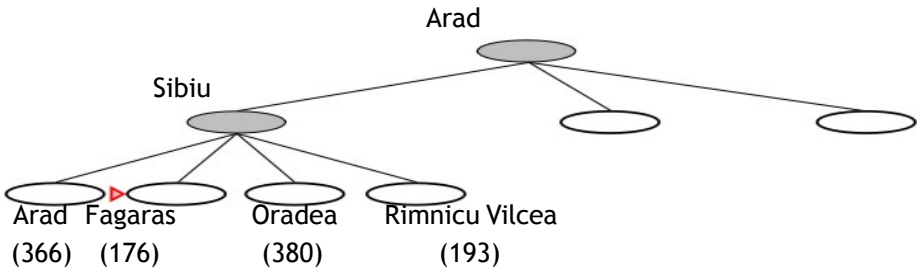
- Utilizar busca gulosa para viajar de Arad até Bucareste.
- Estado Inicial = Arad.

Busca Gulosa



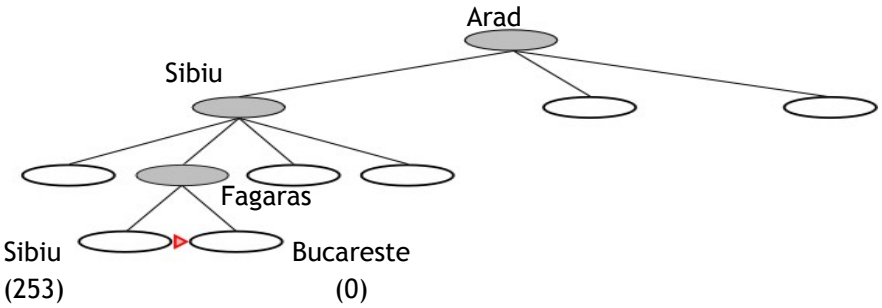
- O primeiro passo de expansão produz:
  - Sibiu, Timisoara e Zerind.
- Busca Gulosa irá selecionar Sibiu.

Busca Gulosa



- Quando Sibiu é expandida obtém-se:
  - Arad, Fagaras, Oradea e Rimnicu Vilcea.
- Busca Gulosa irá selecionar Fagaras.

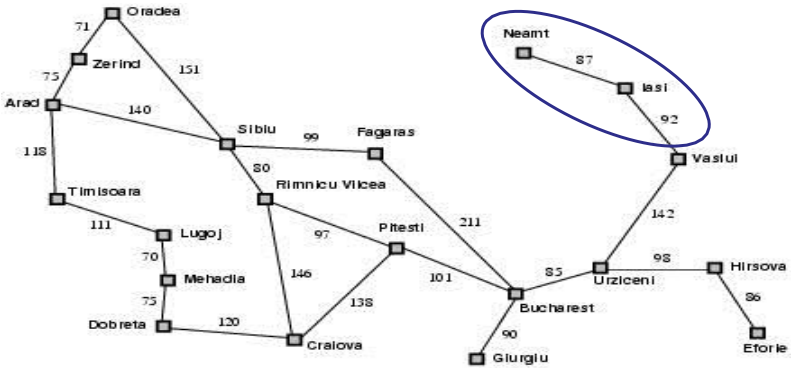
Busca Gulosa



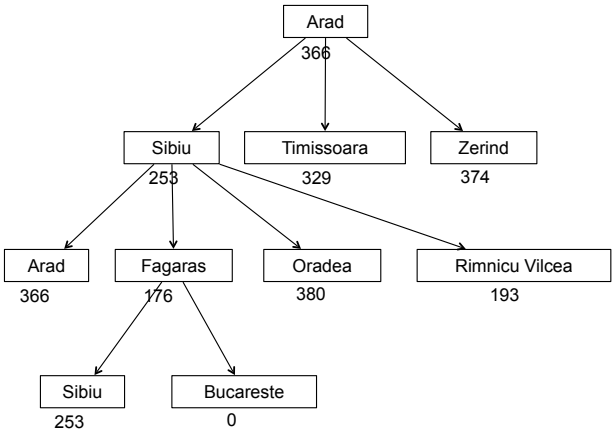
- Expandindo Fagaras obtém-se:
  - Sibiu e Bucareste.
- Objetivo alcançado
  - Porém não é ótimo (veja Arad, Sibiu, Rimnicu Vilcea, Pitesti).

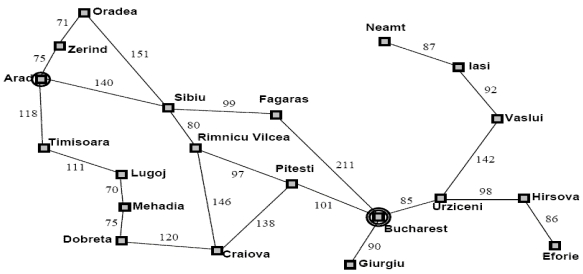
Busca Gulosa

- Completude: NÃO.
  - Assim como busca em profundidade, tenta seguir ao longo de um único caminho até a meta, podendo nunca retornar devido a estados repetidos.
  - Minimizar h(n) pode produzir laços infinitos:
    - E.g., de Iasi para Fagaras fica-se preso indefinidamente entre Iasi e Neamt.



Busca Gulosa





Arad	366	Mehadia	241
Bucareste	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374
Hirsova	151	Urziceni	80

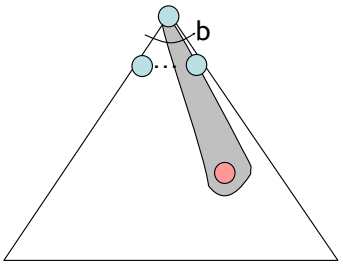
Função Heurística (h): Distância em linha reta

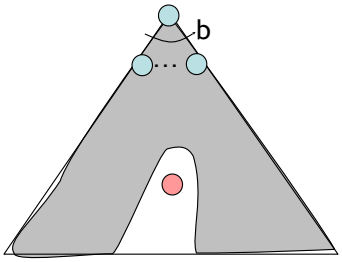
Busca Gulosa

- **Custo de busca mínimo:**
  - No exemplo, não expande nós fora do caminho.
- **Não é ótima:**
  - No exemplo, escolhe o caminho que é mais econômico à primeira vista, via Fagaras.
  - Porém, existe um caminho mais curto via Rimnicu Vilcea.
- **Não é completa:**
  - Pode entrar em laço infinito se não detectar a expansão de estados repetidos.
  - Pode tentar desenvolver um caminho infinito.

Busca Gulosa

- Estratégia: expandir um nó que você acha que está mais próximo de um estado objetivo
  - Heurística: estimativa da distância do objetivo mais próximo para cada estado
- Um caso comum
  - Best-first leva para o objetivo errado
- Pior caso: semelhante a DFS





Busca A*

- Idéia: considerar também o custo de alcançar cada nó candidato n a partir do nó (estado) inicial.
- Função de Avaliação f(n) = g(n) + h(n)
  - g(n) custo do caminho do nó inicial até o nó n.
  - h(n) custo estimado do nó n até o nó objetivo (heurística).
  - f(n) custo estimado total do caminho através de n até o objetivo.

- $g(n) = 0 \rightarrow$ Busca de custo uniforme
- $h(n) = 0 \rightarrow$ Busca gulosa de melhor escolha

Busca A*

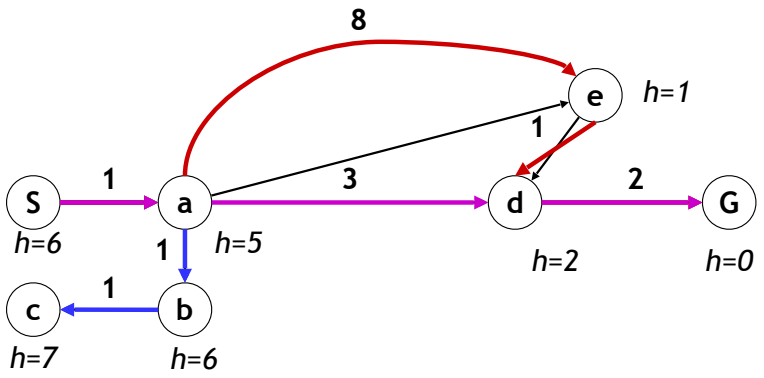
- Busca A* utiliza uma heurística admissível
  - Nunca superestima o custo para alcançar a meta.
  - Ou seja, heurísticas admissíveis são otimistas.

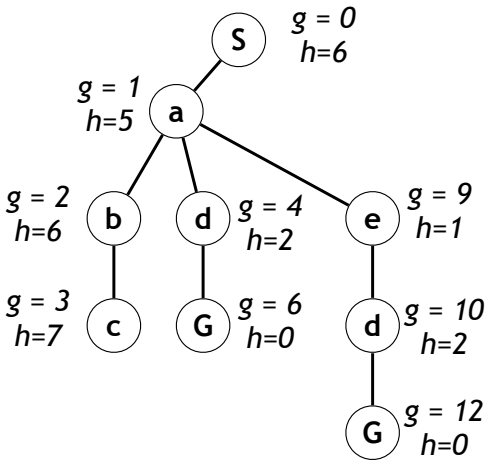
- Formalmente:
  - $h(n) \leq h^*(n)$, $h^*(n)$ é o custo verdadeiro de n até o objetivo.
  - Dado $h(n) \geq 0$, tem-se $h(G) = 0$ para qualquer objetivo G.

- $h_{DLR}(n)$ nunca superestima a distância real da estrada.

Combina Custo Uniforme e Busca Gulosa

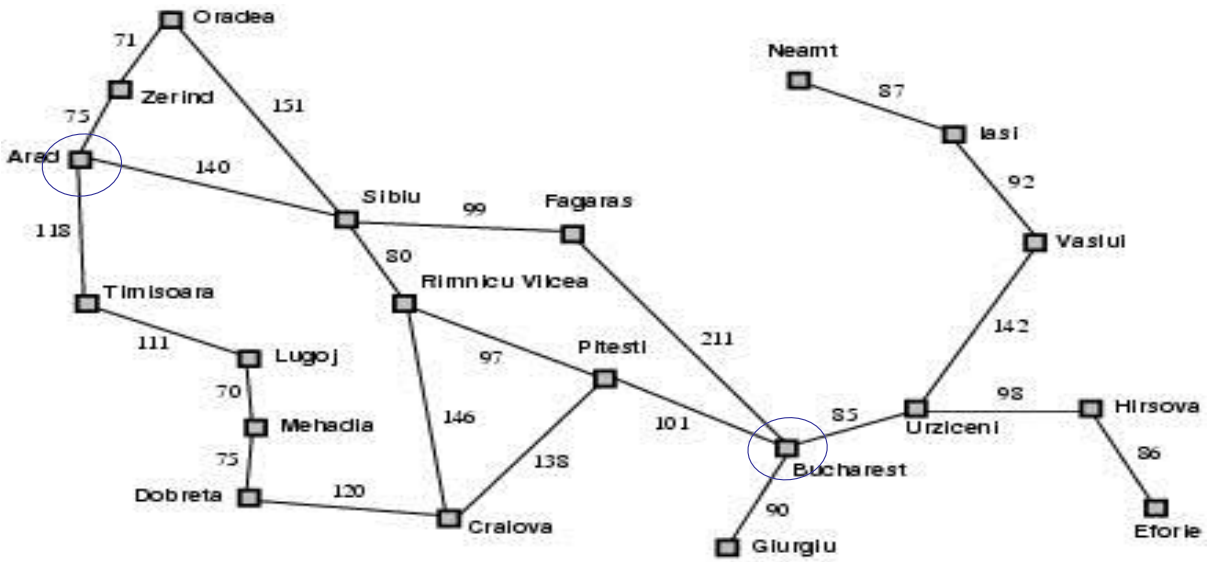
- Custo uniforme ordena pelo custo de caminho g(n)
- Gulosa ordena pela proximidade do objetivo h(n)





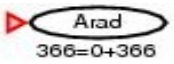
- Busca A* ordena pela soma: f(n) = g(n) + h(n)

Busca A*



Busca A*

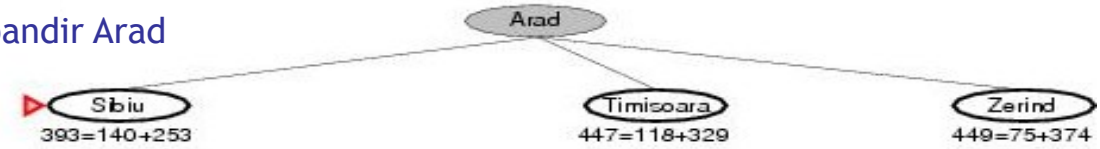
- (a) Estado Inicial



- Encontrar Bucareste partindo de Arad:
- f(Arad) = 0 + h(Arad) = 0 + 366 = 366

Busca A*

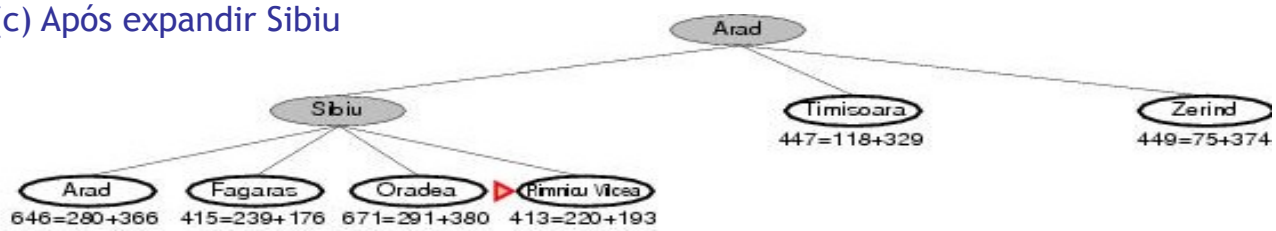
- (b) Após expandir Arad



- Expandindo Arad e determinando f(n) para cada nó:
  - f(Sibiu) = c(Arad,Sibiu) + h(Sibiu) = 140 + 253 = 393.
  - f(Timisoara) = c(Arad,Timisoara) + h(Timisoara) = 118 + 329 = 447.
  - f(Zerind) = c(Arad,Zerind) + h(Zerind) = 75 + 374 = 449.
- Melhor escolha é Sibiu.

Busca A*

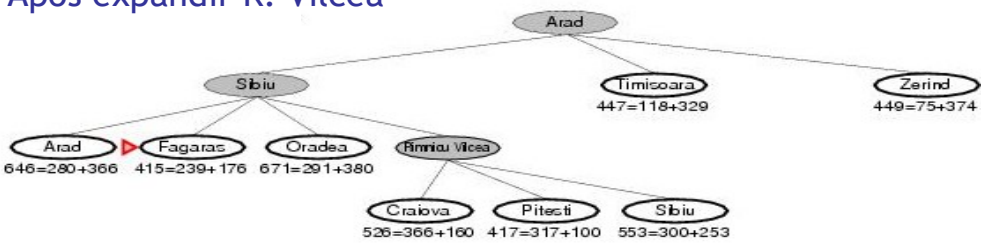
- (c) Após expandir Sibiu



- Expandindo Sibiu e determinando f(n) para cada nó:
  - f(Arad) = g(Arad) + h(Arad) = 280 + 366 = 646.
  - f(Fagaras) = g(Fagaras) + h(Fagaras) = 239 + 176 = 415.
  - f(Oradea) = g(Oradea) + h(Oradea) = 291 + 380 = 671.
  - f(Rimnicu Vilcea) = g(Rimnicu Vilcea) + h(Rimnicu Vilcea) = 220 + 193 = 413.
- Melhor escolha é Rimnicu Vilcea.

Busca A*

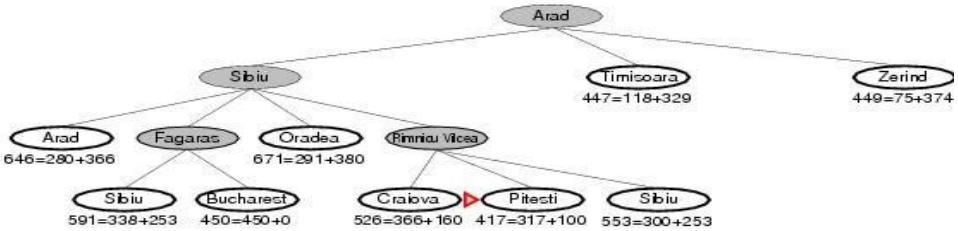
- (d) Após expandir R. Vilcea



- Expandindo Rimnicu Vilcea:
  - f(Craiova) = g(Craiova) + h(Craiova) = 366+160 = 526.
  - f(Pitesti) = g(Pitesti) + h(Pitesti) = 317 + 100 = 417.
  - f(Sibiu) = g(Sibiu) + h(Sibiu) = 300 + 253 = 553.
- Melhor escolha é Fagaras.

Busca A*

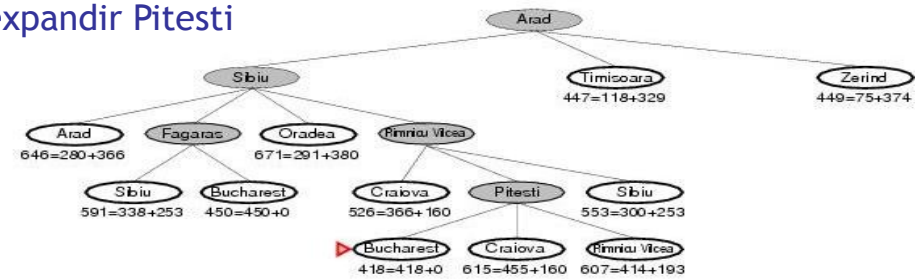
- (e) Após expandir Fagaras



- Expandindo Fagaras:
  - f(Sibiu) = g(Sibiu) + h(Sibiu) = 338 + 253 = 591.
  - f(Bucareste) = g(Bucareste) + h(Bucareste) = 450 + 0 = 450.
- Melhor escolha é Pitesti.

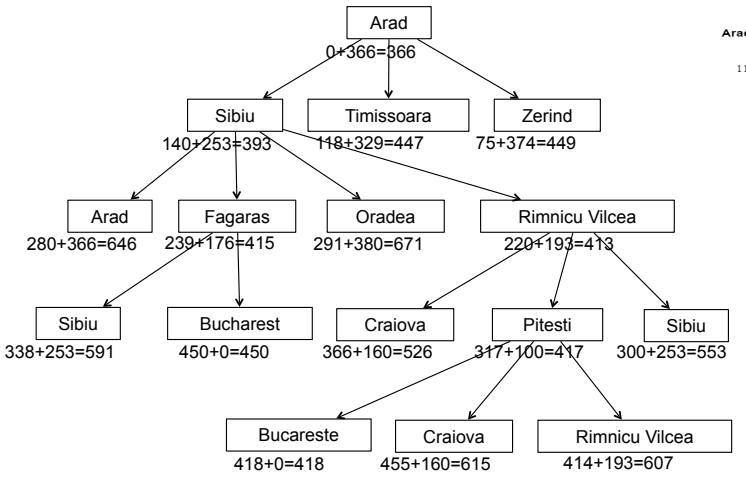
Busca A*

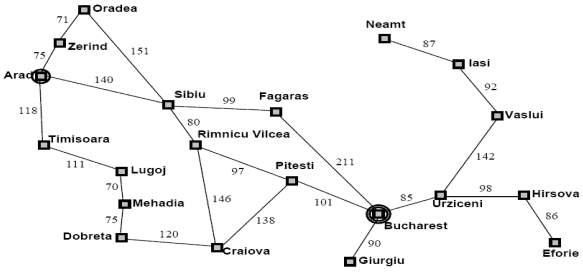
- (f) Após expandir Pitesti



- Expandindo Pitesti:
  - f(Bucareste) = g(Bucareste) + h(Bucareste) = 418 + 0 = 418.
- Melhor escolha é Bucareste.
  - Solução ótima (dado que h(n) é admissível).

Busca A*





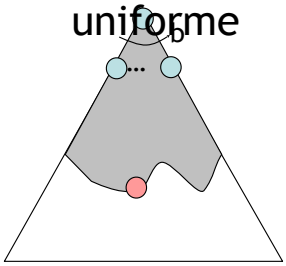
Arad	366	Mehadia	241
Bucareste	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374
Hirsova	151	Urziceni	80

Busca A*

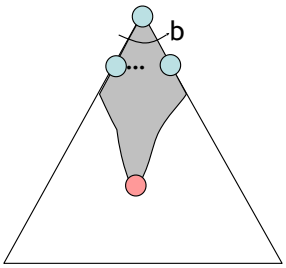
- A estratégia é **completa** e **ótima**.
- **Custo de tempo:**
  - Exponencial com o comprimento da solução, porém boas funções heurísticas diminuem significativamente esse custo.
- **Custo memória:**
  - Guarda todos os nós expandidos na memória.
- Nenhum outro algoritmo ótimo garante expandir menos nós.

Propriedades da Busca A*

Busca de custo uniforme

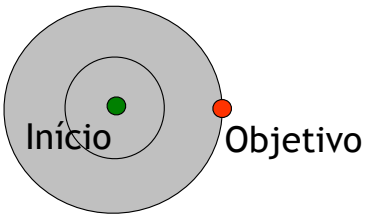


Busca A*

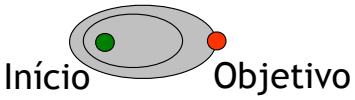


Propriedades da Busca A*

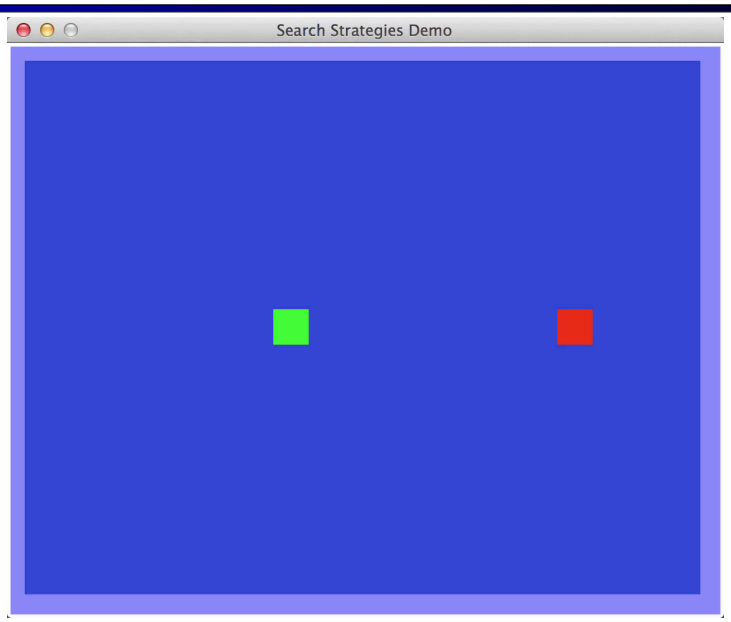
- Busca de custo uniforme expande os nós igualmente em todas as *direções*



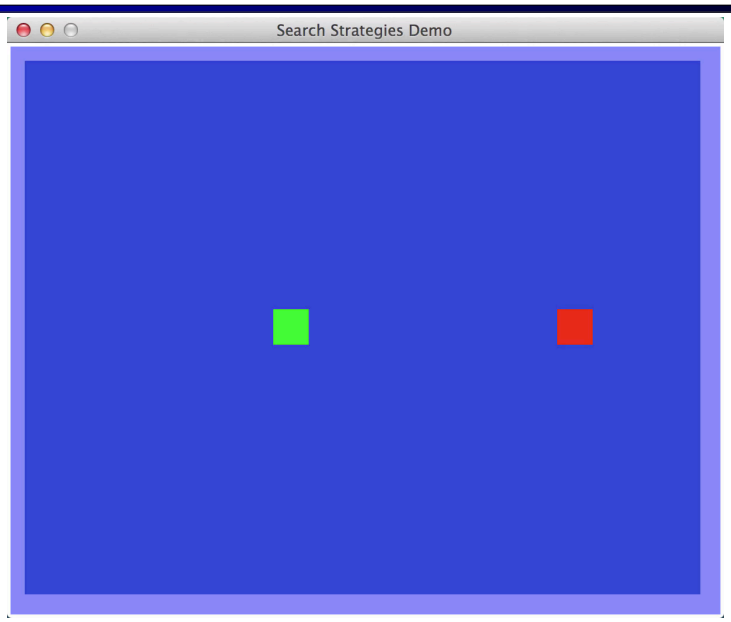
- Busca A* expande principalmente em direção ao objetivo



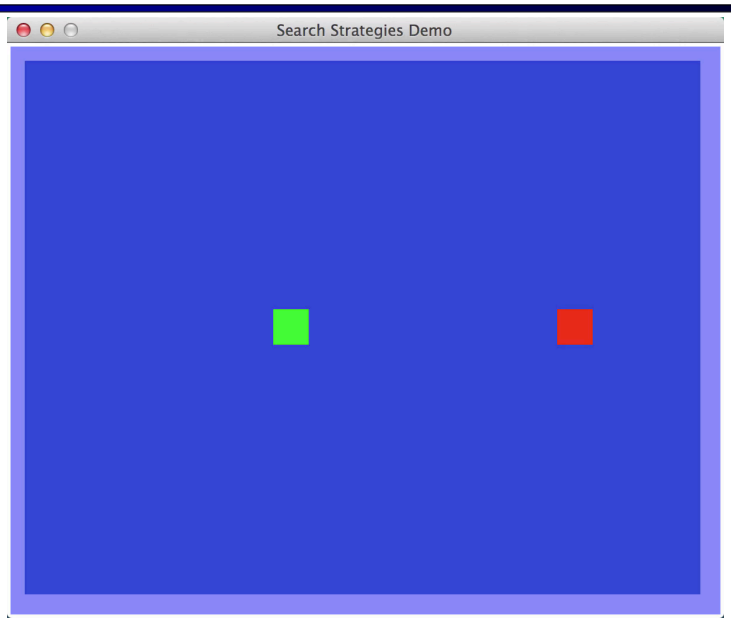
Contornos para busca de custo uniforme (UCS)



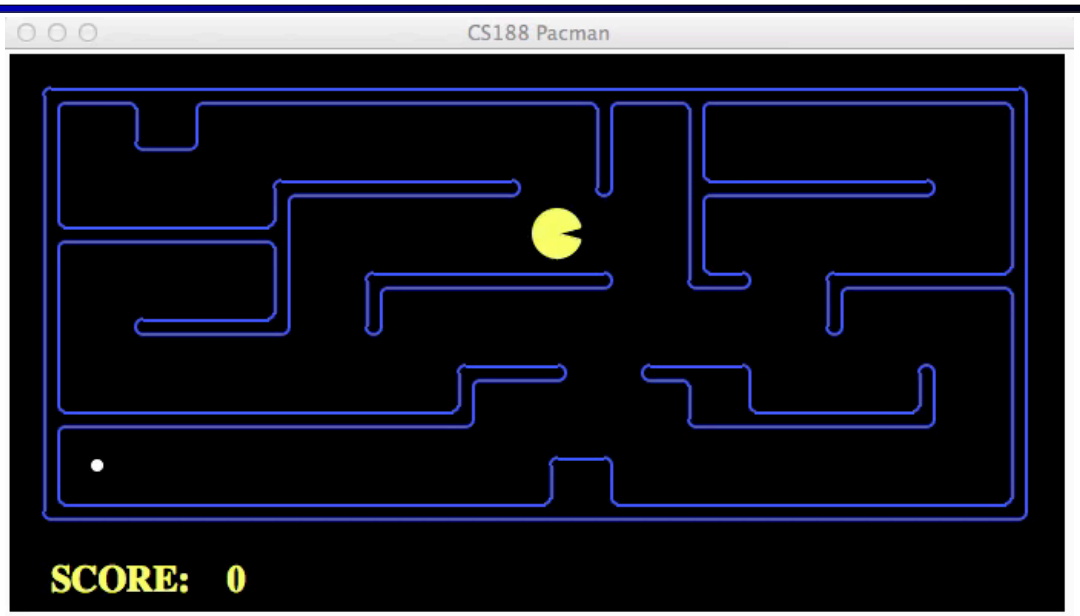
Contornos para a busca gulosa (Greedy)



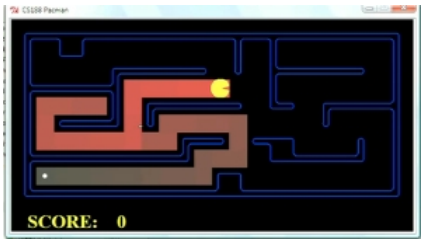
Contornos para a busca A*



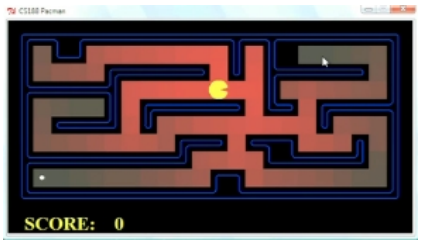
Contornos com Pacman – A*



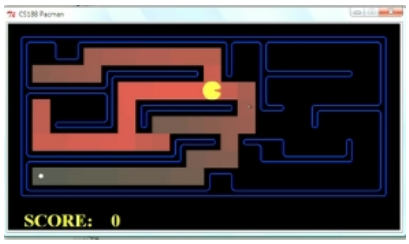
Comparação



Gulosa



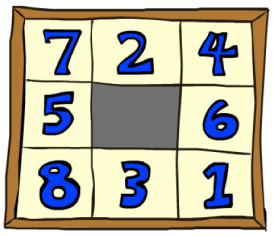
Custo Uniforme



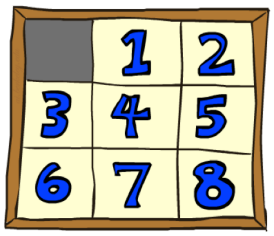
A*

Definindo Heurísticas Admissíveis

- Cada problema **exige** uma função heurística diferente.
- Não se deve superestimar o custo real da solução.
- Como escolher uma boa função heurística para o jogo de 8 peças?

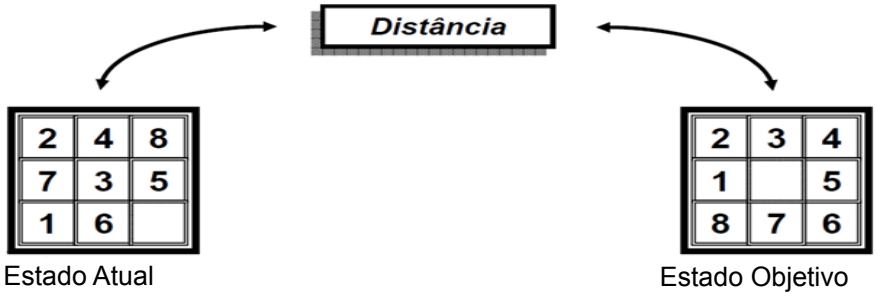


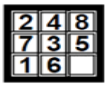
Estado inicial



Estado objetivo

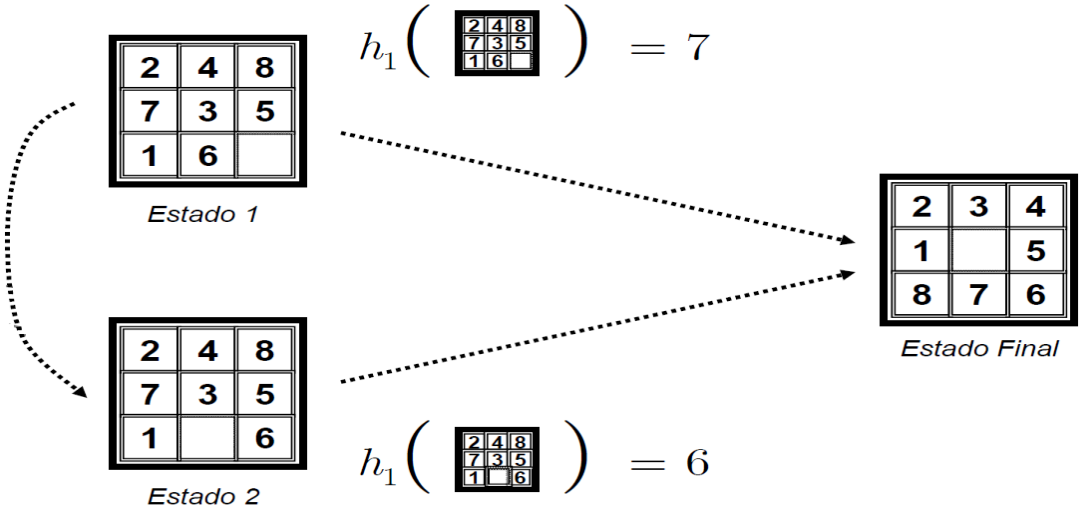
Definindo Heurísticas Admissíveis



$h_1($  $) =$ *A quantidade de peças for a do lugar*

$= 7$

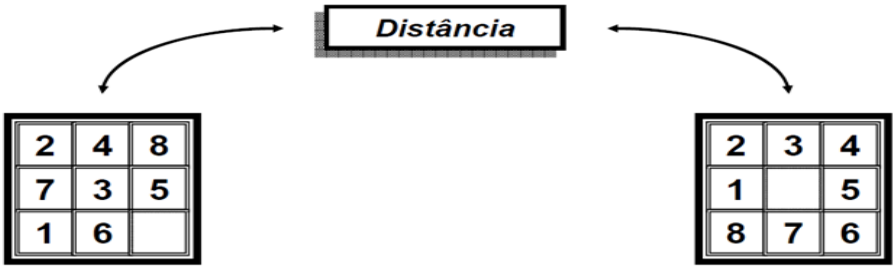
Definindo Heurísticas Admissíveis

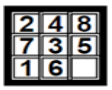


$h_1($ $) = 7$

$h_1($ $) = 6$

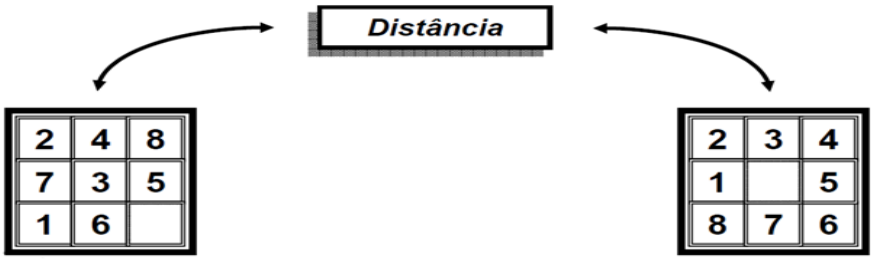
Definindo Heurísticas Admissíveis



h_2 () $= ?$

Outra Heurística?

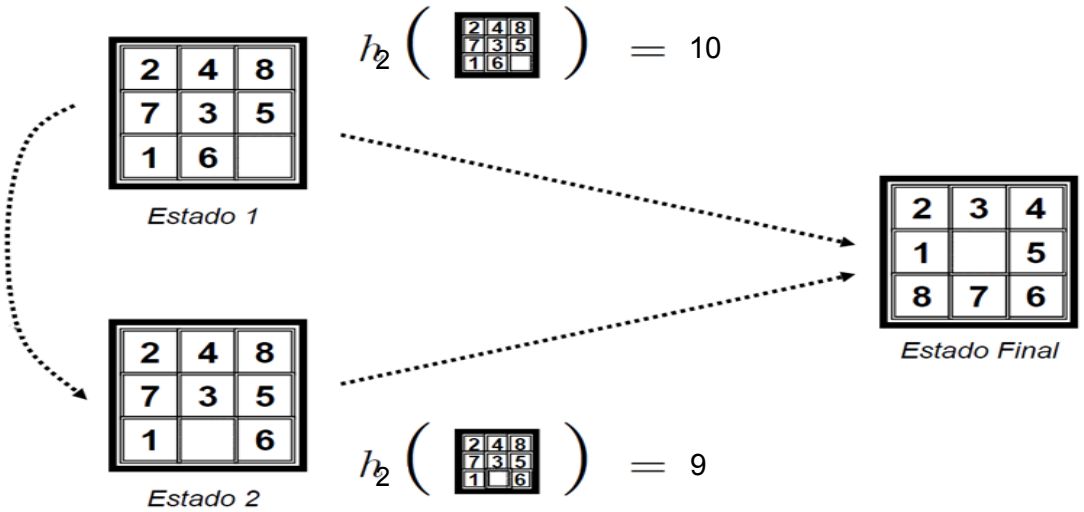
Definindo Heurísticas Admissíveis



h_2 ( ) $=$ Número de movimentos necessários para colocar cada peça no seu lugar

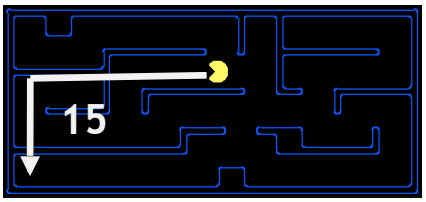
$= 10$

Definindo Heurísticas Admissíveis



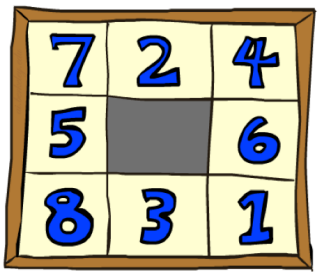
Definindo Heurísticas Admissíveis

- A maioria dos trabalhos em resolver problemas de busca em busca do ótimo é acompanhada de heurísticas admissíveis
- Geralmente heurísticas admissíveis são soluções para *problemas relaxados*, onde novas ações estão disponíveis

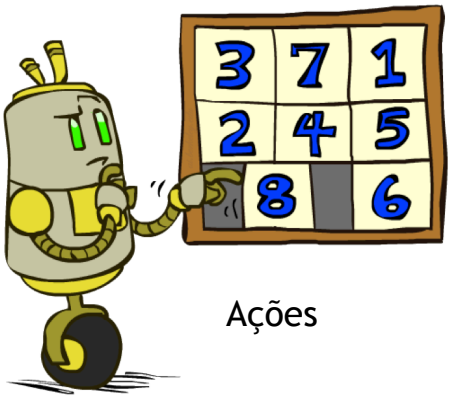


- Heurísticas inadmissíveis frequentemente são úteis também

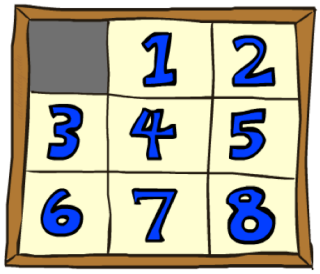
Jogo de 8 peças



Estado inicial



Ações

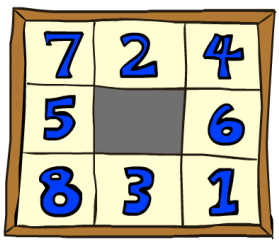


Estado objetivo

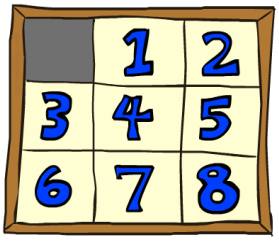
- Quais são os estados?
- Quantos são os estados?
- Quais são as ações?
- Quantos sucessores desde o estado inicial?
- Quais devem ser os custos de movimento?

Jogo de 8 peças: heurística I

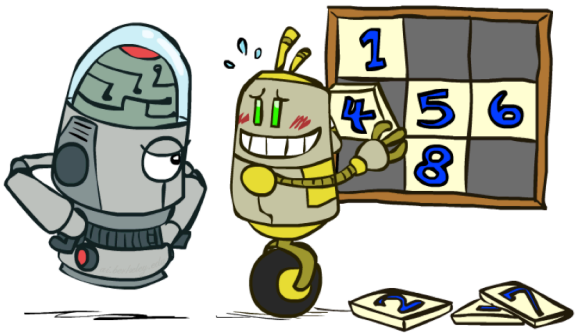
- Heurística: número de peças erradas
- Por que é admissível?
- h(início) = 8
- Esta é uma heurística *relaxada* (versão simplificada do problema original com operadores menos restritivos)



Estado inicial



Estado objetivo

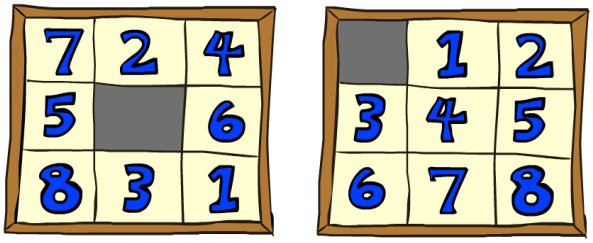


	Média dos nós expandidos quando o caminho ótimo tem...		
	...4 passos	...8 passos	...12 passos
UCS	112	6,300	3.6×10^6
Peças	13	39	227

Estatística de Andrew Moore

Jogo de 8 peças: heurística II

- Se tivéssemos um jogo onde qualquer peça pudesse deslizar em qualquer direção a qualquer momento, ignorando outras peças?
- Distância total de *Manhattan*
- Por que é admissível?
- h(início) =3 + 1 + 2 + ... = 18

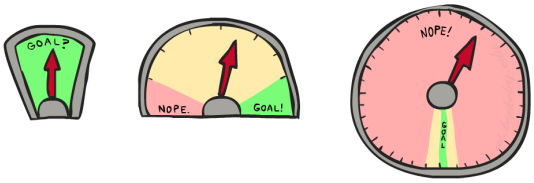


Estado inicial Estado objetivo

	Média dos nós expandidos quando o caminho ótimo tem...		
	...4 passos	...8 passos	...12 passos
Peças	13	39	227
Manhattan	12	25	73

Jogo de 8 peças: heurística III

- E se utilizarmos o custo atual como heurística?
  - Seria admissível?
  - Poderíamos salvar os nós expandidos?
  - Qual o problema nisso?



- Com a busca A*: um "meio termo" entre qualidade da estimativa e trabalho por nós
  - Enquanto a heurística se aproxima do custo real, pode-se expandir poucos nós mas geralmente se executa mais trabalho por nó para computar a heurística

Busca Local e Otimização

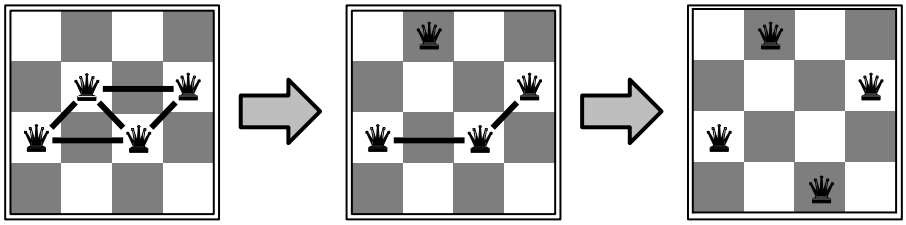
- Anteriormente: exploração sistemática do espaço de busca.
  - Caminho para o objetivo é a solução para o problema.
- Em muitos problemas o caminho é irrelevante: o estado objetivo é a solução
  - Exemplo: encontrar máximos/mínimos de funções unimodais ou multimodais.
- Espaço de estados: conjuntos de configurações completas
  - Escolha uma configuração que satisfaça as restrições (ex.: n-rainhas)
- Algoritmos de natureza distinta podem ser utilizados:
  - Busca Local: mantenha um simples estado atual e tente melhorá-lo
  - Otimização combinatória
  - Computação Natural: inspiração na natureza

Busca Local e Otimização

- Busca Local = armazenamento apenas do estado corrente e movimento para estados vizinhos.
- Vantagens:
  - Utiliza pouca memória (usualmente uma quantidade constante).
  - Frequentemente encontra boas soluções em espaço de estados muito grandes ou mesmo infinitos, nos quais estratégias sistemáticas são inadequadas ou inviáveis.

Exemplo: *n*-rainhas iterativa

- Coloque **n** rainhas em um tabuleiro ***n* × *n*** sem duas rainhas estarem na mesma linha, coluna ou diagonal
- Mova uma rainha para reduzir o número de conflitos



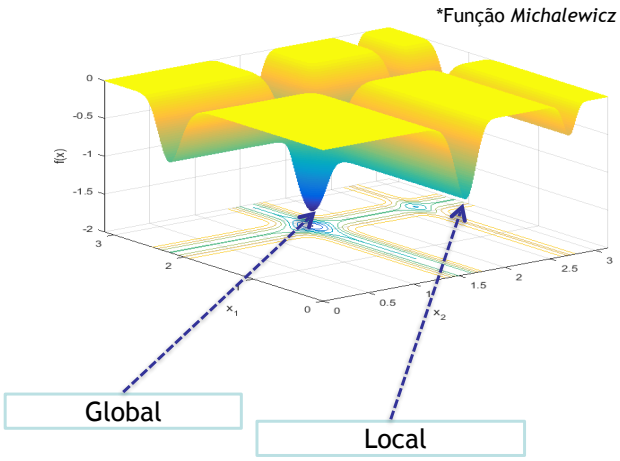
h = 5 **h = 2** **h = 0**

Modelo

- Forma *padrão* de um problema de otimização

$$
\begin{aligned}
& \min f(x) \\
Sujeito: \quad & g(x) \leq 0 \\
& h(x) = 0 \\
& x \in X \subset R^n \\
& \lfloor x \rfloor \leq x \leq \lceil x \rceil
\end{aligned}
$$

*Função *Michalewicz*



Subida da Encosta (*Hill Climbing*)

- Política de busca local cujos movimentos se dão impreterivelmente na direção de valores crescentes (ou não decrescentes) da função objetivo.
  - Termina quando um pico (mínimo ou máximo) é alcançado.
- Move-se apenas para "estados vizinhos" ao corrente.
- O conceito de estado vizinho depende do problema.
- *Hill Climbing* é considerada uma estratégia gulosa.

Subida da Encosta (*Hill Climbing*)

- Diferentes versões:
  - Original (maior passo): move-se para o estado vizinho que proporciona a maior melhoria (redução ou aumento) no valor da função objetivo.
  - *First-Choice*: versão estocástica onde se move para algum estado vizinho sorteado aleatoriamente e que esteja entre aqueles (se existirem) que melhoram o valor corrente da função objetivo.
    - Interrompe a busca após um limite pré-especificado de tentativas de movimento frustradas.
    - Adequando em problemas nos quais a quantidade de vizinhos é grande.
  - *Random-Restart*: Múltiplas buscas *hill climbing* a partir de diferentes estados iniciais gerados aleatoriamente.
    - Adequado em problemas com superfícies de otimização complexas (muitos mínimos ou máximos locais não satisfatórios).

Subida da Encosta (*Hill Climbing*)

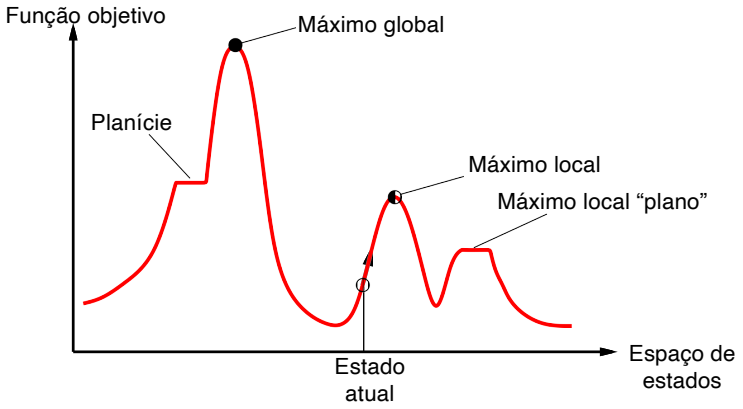
- Com escalar o Everest com muita neblina e sem memória

```
função Subida_da_Encosta(problema) retorna um estado que é um máximo local
  entradas: problema, um problema
  variáveis locais: atual, um nó
                    vizinho, um nó

  atual ← Criar_Nó(Estado_Inicial[problema])
  repita
     vizinho ← um sucessor de atual com valor mais alto
     if Valor[vizinho] ≤  Valor[atual] então retornar Estado[atual]
     atual ← vizinho
  fim
```

Subida da Encosta (*Hill Climbing*)

- Problema: dependendo do estado inicial, pode ficar preso em um máximo local

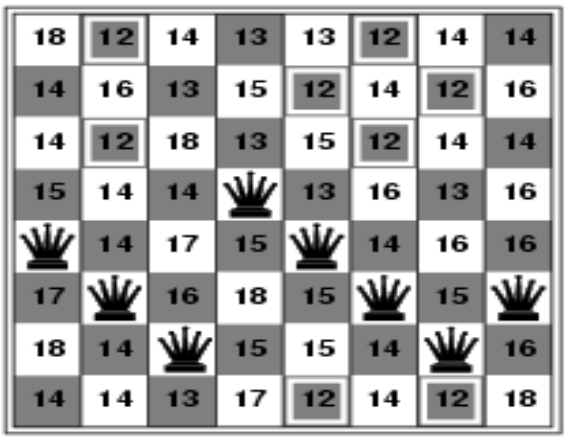


Subida da Encosta: problema das 8 rainhas

- h = números de pares de rainhas que estão se atacando, diretamente ou indiretamente

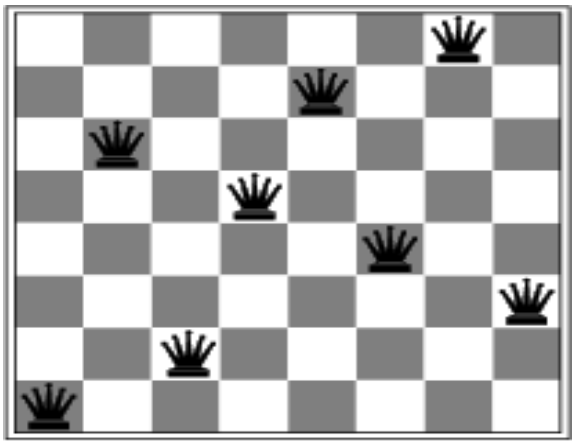
h =

17



Subida da Encosta: problema das 8 rainhas

- h = 1 (mínimo local)



Referências

- Russell, S. J. e Norvig, P.; Inteligência Artificial, 4ª ed., Rio de Janeiro, 2022.
- Klein, D. e Abbeel, P.; CS188 Intro to AI at UC Berkeley. Disponível em: http://ai.berkeley.edu
- Modelo dos *slides* inspirado em: Klein, D. e Abbeel, P.; CS188 Intro to AI at UC Berkeley.
- Aulas do Prof. Edirlei Soares Lima. Inteligência Artificial, PUC-RIO.